

Build a Multi-Step LLM Agent

Individual Assignment • Grok API • Domain of your choice

Purpose

Large language models are not search engines. Their power comes from reasoning — the ability to decompose a problem, act on intermediate results, and refine outputs through successive steps. A single prompt that returns a single response barely scratches the surface of what these systems can do.

This assignment asks you to build something that demonstrates the real capability of LLMs: a multi-step agent where each LLM call produces an output that becomes the input to the next. You will have to think about what information each step needs, what form it should take, and how failures propagate through the chain.

By the end, you will have built a working AI pipeline from scratch, designed prompts for specific sub-tasks, integrated at least one external tool, and reasoned honestly about where your system breaks. These are the skills that matter in applied NLP and AI engineering — not the ability to call an API once.

What You Must Build

You will build a goal-driven agent that accepts a user's input, breaks it into multiple sub-tasks, and processes each sub-task through a dedicated LLM call — where the output of each call feeds directly into the next. The agent must satisfy all of the following requirements.

Multi-step chaining — mandatory

Your agent must make a minimum of four sequential LLM calls. Each call must receive structured output from the previous step as part of its input. A chain of four independent prompts with no data flow between them does not qualify. The dependency between steps must be explicit and traceable in your code.

At least one tool call — mandatory

At least one step in your chain must call an external tool rather than the LLM. A tool is anything that retrieves or computes information the LLM cannot produce from its training alone: a web search, a weather API, a calculator, a file reader, a database query, a code executor. The tool's output must enter the chain as structured data used by a subsequent LLM call.

State across the chain — mandatory

Your agent must maintain a shared state object that accumulates results as the chain progresses. Each step reads from and writes to this state. At the end of the chain, the state contains everything the agent has learned or produced. A simple Python dictionary is sufficient.

Structured final output — mandatory

The agent must produce a structured final output — a formatted report, a JSON object, a filled template, or a clearly organised document. Raw LLM text dumped to the terminal does not qualify. The output should be something a real user could act on directly.

Domain — your choice

There is no fixed domain. Pick a task where multi-step reasoning genuinely adds value over a single prompt — something where the answer depends on gathering information, processing it in stages, and synthesising it into something useful. The domain you choose will be judged on whether it actually benefits from a chain, not on novelty.

Example: Job Application Assistant

The following is one example of what a qualifying agent looks like. You are not required to build this. It is here to show you what the chaining requirement means in practice.

A user pastes a job description and their resume. The agent produces a tailored cover letter through six steps, each building on the last.

Step	LLM Call / Tool	Input	Output (fed to next step)
1	LLM — extract	Raw job description (user paste)	Structured list: required skills, role type, seniority
2	Tool — web search	Company name extracted in Step 1	Company overview: industry, size, recent news
3	LLM — match	Skills list (Step 1) + user resume + company context (Step 2)	Gap analysis: matched skills, missing skills, talking points
4	LLM — draft	Gap analysis (Step 3) + user's tone preference	First draft of cover letter
5	LLM — critique	Draft cover letter (Step 4) + original job requirements (Step 1)	Critique with specific suggestions
6	LLM — refine	Draft (Step 4) + critique (Step 5)	Final polished cover letter

Notice that no single step can be removed without breaking the chain. Step 3 cannot run without the outputs of Steps 1 and 2. Step 5 cannot run without the draft from Step 4 and the

requirements from Step 1. This dependency structure is what distinguishes a pipeline from a collection of independent prompts.

Also notice that Step 2 is a tool call, not an LLM call. The agent retrieves real company information rather than asking the LLM to hallucinate it. That retrieved data then enters the LLM context in Step 3.

Deliverables

All four components are mandatory. Missing any one of them will result in an incomplete submission.

1. Working Code

Submit a GitHub repository link or a .zip file. The code must run without modification given a Grok API key. Include a README that describes: what the agent does, how to install dependencies, how to run it, and what inputs it expects. The chaining logic must be clearly structured in code — a single monolithic function with everything inside it will be marked down.

2. Live Demo

You will run your agent live during the evaluation session. The demo must show the full chain executing end-to-end. Be prepared to pause at any step and explain what input that step received, what the LLM was asked to do with it, and what the output was. You should also show one case where the agent handles an unexpected or malformed input.

Expect these questions during the demo:

- Show me what Step 3 received as input and what it returned.
- What happens if the tool call in Step 2 fails or returns nothing?
- Why did you choose this prompt for this specific step?
- Where does this chain break, and how do you know?

3. Written Report

Maximum 2 pages excluding references. Write in prose, not bullet points. The report must cover:

1. Problem statement — what task does your agent solve, and why does it benefit from multi-step chaining rather than a single prompt?
2. Chain design — what does each step do, what does it receive, what does it produce, and why is it a separate step?
3. Tool integration — what tool did you use, why that tool, and how does its output enter the chain?
4. Limitations — where does the chain fail, what inputs break it, what did you not have time to fix?
5. Reflection — what would you change if you had more time?

4. Prompt Design Explanation

For each LLM step in your chain, include the full system prompt and user prompt you used. Explain why each prompt is shaped the way it is — what constraint or format does it impose, and why does the next step depend on that structure? Show at least one example of a prompt you changed after testing, and explain what you changed and why.

This section can be an appendix to your report. A prompt dump with no explanation will not earn marks.

Evaluation Rubric

All four components are weighted equally. The evaluator will assess each component at three levels: does not meet expectations, meets expectations, and exceeds expectations.

Component	What is Evaluated	Weight
Working Code	Runs without errors; chaining logic is explicit and traceable; repo/zip with README submitted	Equal
Live Demo	Full chain executes end-to-end; student explains each step's input and output live; handles at least one unexpected input	Equal
Written Report	Clear problem statement, chain design rationale, honest limitations, and a genuine reflection	Equal
Prompt Design	Each step's prompt is shown and justified; student explains how output of one step shapes the next	Equal

What exceeds expectations looks like

- Code: the chain is modular — each step is a separate function with clear inputs and outputs; error handling exists at the tool call; the README explains the chain structure, not just how to run it.
- Demo: the student can explain any step's prompt and output without preparation; they can describe a failure mode they discovered during testing and show what it looks like.
- Report: the chain design section explains why each step is separate, not just what it does; limitations are specific and honest, not vague; the reflection identifies something non-obvious.
- Prompt design: the student shows prompt iteration — a version that did not work, what it produced, what they changed, and why the new version works better.

Technical Setup

You must use the Grok API for all LLM calls. For tool calls, you may use any free-tier external service — a search API (e.g. Serper, Tavily, DuckDuckGo), a weather API, a public data API, a local file reader, or a Python library that computes something (e.g. a calculator, a PDF parser).

Recommended stack:

- Language: Python (preferred) or JavaScript/TypeScript
- Interface: command-line, Gradio, or Streamlit — keep it simple; the interface is not graded
- State: a Python dictionary that each step reads from and writes to
- Output: write the final structured output to a file (JSON, Markdown, or plain text)

Do not use LangChain, LlamaIndex, or agent frameworks. Build the chain yourself. If you use a framework, you will be asked during the demo to explain every abstraction it provides. If you cannot, it will reflect in your marks.

Academic Integrity

You may use LLMs to help debug code, generate boilerplate, or explore prompt ideas. You may not submit a chain that was designed entirely by an AI. The chain structure, the choice of steps, the prompt design, and the tool integration must reflect your own reasoning.

If you used an LLM significantly during development, note it in your reflection: what you asked, what it got wrong, and what you had to correct. This is not penalised — it is informative.

Questions? Reach out before the demo, not during it.